

Polymorphic Type Inference

Algorithm (SML / ML)

1. Assign fresh type variables to every parameter.
2. Recurse down AST as in normal type checking.
3. On the way back up, **unify** the type you have with the type you expect.

Unification

unify(T₁, T₂): returns a **substitution** mapping type vars to types making $T_1 = T_2$, or **fails**.

Base cases:

- Type var with type: return mapping
- Same type with itself: return $\emptyset$
- Constructor mismatch: **FAIL**

Recursive case:

```
unify(T1*T2, S1*S2):
  sub1 = unify(T1, S1)
  T2' = apply(sub1, T2)
  S2' = apply(sub1, S2)
  sub2 = unify(T2', S2')
  return combine(sub1, sub2)
```

Key: Apply sub from left before unifying right.

Occurs Check

When mapping 'b to type T , 'b **cannot appear inside** T . Otherwise infinite type.

Ex: `fun f(x) = f` unifies 'b with 'a $\rightarrow$ 'b — fails occurs check.

Circumvent with datatypes:

```
datatype 'a hungry = F of 'a -> 'a hungry
fun f(x) = F(f) (* 'a -> 'a hungry *)
```

Practice Final Q1

```
Infer: fun f(w,x) = case x of [] => [] | y::z => w(y)::f(w,z)
Result: f : ('h -> 'j) * 'h list -> 'j list
fun f(x) = f — fails occurs check.
```

Subtyping

- **Covariance:** `read` $\Rightarrow$ same direction
- **Contravariance:** `write` $\Rightarrow$ opposite direction
- **Invariant:** `read+write` $\Rightarrow$ no subtyping

Functions: covariant in return, contravariant in arg.

If `Lion <: Cat`: `Lion list <: Cat list` (covariant); `Lion ref <: Cat ref` (invariant); `Cat  $\rightarrow$  int <: Lion  $\rightarrow$  int` (contra in arg).

SML: 'a=any type, ''a=equality types. Complex constraints: **functors**.

MIPS Registers & Calling Convention

Reg	Name	Purpose
r0	zero	Hardwired to 0
r1	AT	Reserved for assembler
r2-3	v0-v1	Return values
r4-7	a0-a3	Arguments (≥ 4 on stack)
r8-15	t0-t7	Caller-saved temps
r16-23	s0-s7	Callee-saved
r24-25	t8-t9	Caller-saved temps
r26-28		Kernel/GP
r29	sp	Stack pointer
r30	fp	Frame pointer
r31	ra	Return address

~25 usable general-purpose registers.

Function Prolog

1. SP $\leftarrow$ frame_size
2. Save old FP
3. Set new FP = SP + frame_size
4. Save callee-saved S regs used
5. Save RA (only if non-leaf)

Function Epilog

1. Restore S regs
2. Restore RA
3. Reset SP (possibly via FP)
4. Restore old FP
5. `jr $ra`

Caller's Responsibilities

Before: save T regs, args to a0-a3 (extras on stack), jal. **After:** restore T regs.

Why Frame Pointer Matters

- FP is **fixed**; SP changes with nested calls, `alloca()`, VLAs
- Vars: FP+offset (stable) vs SP+offset (varies)
- Debugging/stack unwinding uses FP chain
- **Can omit FP** when no dynamic stack alloc

Static Links & Escaping Variables

Static Links

Each frame stores a pointer to the frame of its **statically enclosing function**.

To access a variable N levels out: follow N static links.

In Tiger: SL passed as **implicit first arg** (a0).

SL $\neq$ old FP: Old FP follows dynamic call chain; SL follows lexical nesting.

Practice Final Q2 – Static Links

Nesting: top has odd, f; inside f: g, h.

- f calls g: pass **FP** (child)
- g calls h: pass **MEM(FP)** (sibling)
- g calls itself: pass **MEM(FP)** (sibling)
- h calls f: pass **MEM(MEM(FP))** (2 up)
- g calls odd: pass **MEM(MEM(FP))** (2 up)

Rule: **child** $\rightarrow$ own FP. **sibling/reursive** $\rightarrow$ own SL. **outer** $\rightarrow$ follow SL chain outward.

Escaping Variables

Cannot be placed in a register:

- Tiger: used in a nested function
- C: address taken (`&x`)

Must live in stack frame. **Escape analysis:** track declaration depth; if referenced deeper, mark escaping.

Intermediate Representation (IR)

Why IR?

L langs $\times I$ ISAs $\rightarrow L-I$ compilers. With IR: $L + I$.

AST vs IR vs Assembly

	AST	IR	Asm
Structure	Tree	Tree	Linear
Functions	Nested	Fragment list	Top-level
Variables	Named/scoped	∞ temps	~25 regs
Types	int, string...	32-bit words	byte/word
Cond branch	if/while	2-target CJUMP	1-target+fall

IR Statements (side effects)

- **SEQ(s1, s2):** do s1 then s2
- **LABEL(l):** declare jump target
- **JUMP(e, labels):** unconditional jump
- **CJUMP(relop, e1, e2, lt, lf):** conditional; **two targets**
- **MOVE(dst, src):** dst must be TEMP or MEM
- **EXP(e):** eval for side effects, discard

IR Expressions (produce value)

- **BINOP(op, e1, e2):** arith/logical
- **MEM(e):** in MOVE dst=store; else=load
- **TEMP(t):** register from infinite pool
- **ESEQ(s, e):** do s, return e
- **NAME(l):** label reference
- **CONST(n):** integer constant
- **CALL(f, args):** function call

Ex / Nx / Cx Wrappers

- **Ex(exp):** expression (values)
- **Nx(stm):** statement (effects)
- **Cx(lbl $\times$ lbl $\rightarrow$ stm):** condition fn

Conversions:

- **unEx(Ex(e)) = e**

- **unEx(Nx(s))** = ESEQ(s, CONST 0)
- **unEx(Cx(f))**: alloc r; r=1; f(tl,fl); at fl: r=0; at tl: result = ESEQ(..., TEMP r)
- **unCx(Ex(e))**: CJUMP(e≠0, t, f)

Practice Final Q3 – IR Translation

```
x→InReg t1, y→InReg t2, z→InFrame -4, a→InReg t3.
y := 1+x: MOVE(TEMP t2, BINOP(PLUS, CONST 1, TEMP t1))
a[y] := f(z):
MOVE(MEM(BINOP(PLUS, TEMP t3,
  BINOP(TIMES, TEMP t2, CONST 4))),
  CALL(LABEL(f), [MEM(BINOP(PLUS, FP, CONST -4))]))
```

Translating Control Flow to IR

If-Then-Else

```
translate_if(cond, then_exp, else_exp):
c'=unCx(cond), t'=unEx(then), e'=unEx(else)
r=newTemp, lt/lf/le = newLabels
ESEQ(SEQ(c'(!t,lf), LABEL lt,
  MOVE(TEMP r, t'), JUMP(NAME le,[le]),
  LABEL lf, MOVE(TEMP r, e')),
  LABEL le, TEMP r)
```

While Loop (Optimized)

```
JUMP(NAME L_test, [L_test])
L_body: ... body ...
L_test: CJUMP(cond, L_body, L_done)
L_done:
```

1 branch/iter (saves $\sim N-1$ vs naive 2/iter).

Break: JUMP(NAME(done.label)). Track with label option.

For Loop – Max-Int Edge

Tiger: for i:=low to high (inclusive). If high=maxint, i++ overflows.

```
i:=low; h:=high
if i<=h goto L_body else L_done
L_body: ... body ...
  if i<h then (* STRICT, BEFORE incr *)
    i:=i+1; goto L_body
  else goto L_done
L_done:
```

Arrays

Layout: [size — elem0 — elem1 — ...]. Return ptr to elem0; size at offset -4.

Index: MEM(base + idx×4)

Create: malloc (size+1)×4, store size, incr ptr by 4, fill values.

Bounds: $0 \leq \text{idx} < \text{size}$. Eval idx into temp first.

Records

Create: malloc, MOVE each field to MEM at offset.

Access: MEM(base + offset). No bounds check.

Switch/Case Strategies

- **If-else:** $O(n)$ **Binary search:** $O(\log n)$
- **Jump table:** $O(1)$, for dense cases
- **Hybrid:** binary search → dense jump tables

Why JUMP takes $\text{exp} \times \text{label list}$: jump tables compute

target dynamically.

Critical Rule

Never reuse an IR sub-tree. Dup labels = asm error; dup side effects = bugs. Eval into temp first.

Purifying & Linearizing

Removing ESEQ

Pull side effects out of expressions to top level.

- $\text{ESEQ}(s1, \text{ESEQ}(s2, e)) \rightarrow \text{ESEQ}(\text{SEQ}(s1, s2), e)$
- $\text{BINOP}(+, \text{ESEQ}(s, e1), e2)$:
 - If independent: $\text{ESEQ}(s, \text{BINOP}(+, e1, e2))$
 - Else: $\text{ESEQ}(\text{SEQ}(\text{MOVE}(t, e1), s), \text{BINOP}(+, t, e2))$

Independence needs **alias analysis** (undecidable). Conservative: always use temp.

Hoisting Function Calls

Nested calls clobber shared regs. Pull to top:

```
t1=g(x); t2=h(y); result=f(t1,t2)
```

Args evaluated **left to right** in Tiger.

Alias Analysis

Answers: Yes/No/Maybe. **Undecidable.** Approaches: (1) conservative, (2) type-based, (3) allocation-site. Never false negatives.

Linearizing (Block Ordering)

Goal: false branch of every CJUMP falls through. **DFS** $\gg$ **BFS**.

notRel: negate relops (= to $\neq$, < to $\geq$) when true branch must fall through.

Instruction Selection

Maximal Munch Algorithm

1. Look at root of IR subtree
2. Find **largest subtree** matching one asm instruction
3. Recurse on remaining children
4. Emit instruction, return result register

Key MIPS patterns:

- $\text{MEM}(\text{BINOP}(+, e, \text{CONST } c)) \rightarrow \text{LW rd}, c(\text{rs})$
- $\text{BINOP}(+, e, \text{CONST } c) \rightarrow \text{ADDI rd}, \text{rs}, c$
- $\text{MOVE}(\text{MEM}(\dots), e) \rightarrow \text{SW}$
- $\text{CONST } c \rightarrow \text{LI rd}, c$
- $\text{BINOP}(*, e1, e2) \rightarrow \text{MUL (no mul-imm)}$

Output Data Types

- **OPER**: asm string + src/dst lists + jump targets
- **MOVE**: reg-to-reg (1 src, 1 dst)
- **LABEL**: label marker

Jump list: NONE=fall through; SOME(list)=targets; SOME($\square$)=end of fn (JR RA).

JAL: list **ALL caller-saved** as dsts (t0–t9, a0–a3, v0, ra).

Liveness Analysis

Live: value **may be read** before overwritten. **Dead**: will be written before read.

False positives (say live when dead) are safe.

Data Flow Equations

- **def(B)**: vars written in B
- **use(B)**: vars read before any def in B

Liveness Equations

```
live-in(B) = (live-out(B) - def(B)) ∪ use(B)
live-out(B) =  $\bigcup_{S \in \text{succ}(B)} \text{live-in}(S)$ 
```

Backward, union over successors, **least** fixed point (start $\emptyset$).

Process blocks in **reverse order** for efficiency.

Within a Basic Block

Start with live-out at bottom. Walk **backwards**: writes kill, reads generate.

Instruction-Level Liveness

```
x=q+a; y=c*3; z=5. live-out={x, y, u}.
After z=5: {x, y, u}
After y=c*3: {x, u} - {y} ∪ {c} = {x, c, u}
After x=q+a: {c, u} - {x} ∪ {q, a} = {q, a, c, u}
```

Fixed-Point Iteration

Init all sets to $\emptyset$. Recompute until stable. Finite vars + sets only grow = convergence.

Register Allocation (Graph Coloring)

Interference Graph

Nodes=temps. **Interference edges** (solid): live at same time. **Move edges** (dashed): want to coalesce.

Build: For each write of t , add interference with all live-out temps (except t). For MOVE: add move edge instead. k = physical registers. Goal: k -color the graph.

Algorithm: **Simplify**→**Coalesce**→**Freeze**→**Spill**

1. Simplify

Node with degree $< k$ and **no move edges**: remove, push on stack. Move edges do NOT count toward degree.

2. Coalesce

Merge two move-related nodes.

Coalescing Heuristics (either $\Rightarrow$ safe)

Briggs: Combined node has $< k$ neighbors of **non-trivial degree** ($\geq k$).
George (A into B): Every neighbor of A either already interferes with B, or has trivial degree ($< k$).

After coalesce $\rightarrow$ back to Simplify.

3. Freeze

Give up on a move edge. Prefer edges where ≥ 1 node has trivial degree. Back to Simplify.

4. Potential Spill

Pick node, remove, push as potential spill. Back to Simplify.

5. Select (Rebuild)

Pop stack, assign colors. Simplified = guaranteed. Potential spills = may fail. Pick lowest available color.

6. Actual Spill

Can't color $\Rightarrow$ insert loads/stores, **restart from scratch**.

Pre-colored Temps

Fixed regs (e.g. temp100=a0). All interfere with each other. **Never simplified/spilled**. Base case = only pre-colored remain.

Temp live across JAL $\Rightarrow$ interferes with all caller-saved $\Rightarrow$ naturally gets s-register.

Practice Final Q4

3-reg machine. (1) Simplify b. (2) Coalesce p,q. (3) Simplify pq. (4) Freeze x,c. (5) Simplify x,y,a,z,c. Assign colors bottom-up.

Data Flow Analysis Framework**Master Table**

Analysis	Dir	Meet	Init
Avail Exprs	Fwd	$\cap$ pred	Greatest (full)
Reaching Defs	Fwd	$\cup$ pred	Least ($\emptyset$)
Liveness	Bwd	$\cup$ succ	Least ($\emptyset$)
Very Busy	Bwd	$\cap$ succ	Greatest (full)

Rule: $\cap \Rightarrow$ greatest fixed pt (start full). $\cup \Rightarrow$ least fixed pt (start $\emptyset$).

Available Expressions

$x \text{ op } y$ is **available** if computed on **ALL** paths and operands unmodified since.

$$\text{AE-in}(B) = \bigcap_{P \in \text{pred}} \text{AE-out}(P)$$

$$\text{AE-out}(B) = (\text{AE-in}(B) - \text{Kill}) \cup \text{Gen}$$

Gen: exprs computed in B (operands not killed). **Kill:** exprs using vars defined in B.

Drives: **CSE**. Must start with universal set (starting $\emptyset$ breaks cycles).

Reaching Definitions

Def d : “ $x := \dots$ ” reaches use u if path exists with no redef of x .

$$\text{RD-in}(B) = \bigcup_{P \in \text{pred}} \text{RD-out}(P)$$

$$\text{RD-out}(B) = (\text{RD-in}(B) - \text{Kill}) \cup \text{Gen}$$

Gen: defs in B. **Kill:** other defs of same var.

Drives: **const prop**, **copy prop**, **dead code elim**, **live range split**, **uninit var detection**.

Practice Final Q5 – Reaching Defs

a in instr 4: defs **1, 8**. a in instr 8: defs **1, 8, 13**. b in instr 6: defs **2, 11**.

Very Busy Expressions

Definitely computed on all forward paths before operand modified. $\text{VBE-out}(B) = \bigcap_{S \in \text{succ}} \text{VBE-in}(S)$

Drives: **code hoisting**. Also LICM safety check.

Practice Final Q5 – Matching

CSE $\rightarrow$ Avail Exprs. Fwd/union $\rightarrow$ Reaching Defs.
Def/use web $\rightarrow$ Reaching Defs. Bwd/intersect $\rightarrow$ Very Busy.

Compiler Optimizations

CSE: If expr is available, reuse it. Different paths $\rightarrow$ fresh temp.

Const Prop/Folding: Only reaching def is $x=c \Rightarrow$ replace with c . $0+q \rightarrow q$, $2+3 \rightarrow 5$.

Copy Prop: Only reaching def is $x=y \Rightarrow$ replace x with y .

Dead Code Elim: Result unused + no side effects $\Rightarrow$ remove.

Induction Var Elim: i only used as $4 \times i$; replace with $t += 4$ per iteration. Can eliminate i if comparison $i \geq j$ becomes $t_i \geq t_j$.

LICM: All defs reaching operands from outside loop $\Rightarrow$ hoist. **Danger:** zero-trip loops. Fix: **while** $\rightarrow$ **if{do-while}**.

Live Range Splitting: Connected components of def-use chains via reaching defs $\rightarrow$ separate temps $\rightarrow$ lower degree.

Strength Reduction: $4 \times i \rightarrow i \ll 2$; mul by 3 $\rightarrow$ shift+add.

Domination & Loop ID**Domination**

A **dominates** B if **all** paths from start to B pass through A . Self-dominates.

$$\text{Dom}(B) = \{B\} \cup \bigcap_{P \in \text{pred}} \text{Dom}(P)$$

Entry: $\text{Dom}(\text{entry}) = \{\text{entry}\}$. Intersection $\Rightarrow$ greatest fixed pt (init universal).

Immediate Dominator

$\text{idom}(B)$ = closest strict dominator. Path root $\rightarrow$ B in dom tree = all dominators.

Back Edges & Natural Loops

Back edge: $E \rightarrow S$ where S **dominates** E . Each defines a natural loop.

NOT a loop: cycle where target doesn't dominate source (goto spaghetti).

Practice Final Q6 – Domination

10-node CFG. Back edges: $3 \rightarrow 3$ (self-loop), $8 \rightarrow 4$.
 $4 \rightarrow 1$ is **NOT** a loop: 1 doesn't dominate 4 (path $0 \rightarrow 2 \rightarrow 3 \rightarrow 4$ bypasses 1).
 idom tree: 0 root; 0 idom of 1,2,4; 2 idom of 3; 4 idom of 5; 5 idom of 6,7,8; 8 idom of 9.

Garbage Collection

Garbage: unreachable allocated memory. Precise detection is **undecidable**; approximate by reachability.

Reference Counting

Each obj has count. Inc on new ref, dec on removal. Free when 0 (recursive).

$p=q$: (1) $q \neq \text{null}$: $q.\text{cnt}++$; (2) $p \neq \text{null}$: $p.\text{cnt}--$, if 0 free; (3) $p=q$.

Pros: Simple. **Cons:** Slow ($\sim 10x$), needs atomic ops in MT (~ 50 cyc), **can't collect cycles**.

Mark and Sweep

Root set: regs, stack, globals. **Mark:** DFS, mark reachable. **Sweep:** walk heap, free unmarked, clear marks.

Cost: $O(\text{live} + \text{dead})$. Stop-the-world.

Pointer ID: object header **bitmasks**, stack frame bitmasks, PC-keyed register bitmasks.

Stop and Copy

Heap in 2 halves. Allocate linearly (bump ptr). When full: copy live to other half via DFS. Leave **forwarding ptrs**. Swap.

Pros: $O(\text{live})$ only, no fragmentation, fast alloc, good

locality. **Cons: wastes half heap.**

Generational GC

Infant mortality: most objects die young.

- **Gen 0** (~256KB): stop-and-copy, frequent, fast, fits L2 cache
- **Gen 1** (~4MB): mark-and-sweep
- **Gen 2** (~6GB): mark-and-sweep, rare

Promote after surviving N collections (2-bit counter).

Old→young ptrs: mprotect() marks old pages read-only; writes trigger SIGSEGV to record dirty pages for root set.

Practice Final Q7 – GC

3 advantages: Fast alloc (S&C), locality (S&C), no double frees, no dangling ptrs, no leaks.

Tradeoffs: RefCount=high per-op overhead, 1 word/obj. M&S= $O(\text{live}+\text{dead})$, 1 word/obj. S&C= $O(\text{live})$ but **wastes half**.

Pointer ID: (1) compiler type info/bitmasks, (2) conservative, (3) tagging (1 bit/word).

Compiling OO Languages

Polymorphism Types

1. **Ad-hoc (overloading):** name mangling
2. **Parametric:** C++ specialization (sep code per type) vs Java erasure (compile as Object)
3. **Subtype:** dynamic dispatch via vtables

VTables

Each obj has **one ptr** to shared vtable. Dynamic dispatch:

```
v = load(p+0) // vtable ptr
f = load(v+offset) // method ptr
a0 = p; jalr f // call with this
```

Cost: 2 extra loads vs static dispatch.

Sub-object Rule

Subclass contains superclass sub-object at **same offsets**. Superclass code works unchanged on subclass ptrs.

Multiple Inheritance (C++)

Sub-objects stacked. **Two vtable ptrs**. Non-primary parent needs pointer adjustment. **Thunks:** fixup code adjusts **this** before calling overridden method.

Diamond & Virtual Inheritance

Shared base appears **once** at end. Vtable stores **offset** to shared base $\Rightarrow$ extra indirection.

JIT (Java)

javac $\rightarrow$ bytecode $\rightarrow$ interpret $\rightarrow$ compile hot code to native $\rightarrow$ recompile dynamically. Profile-guided optimization.

Miscellaneous

procEntryExit 1/2/3: prolog/epilog, JR RA (SOME([])), **view shift** (args from a0-a3/stack to

temps).

S-reg strategy: Try 0 s-reg, if fails try 1, etc. Save only what's needed.

Infeasible paths: Data flow may report false liveness on arithmetically impossible paths.

Instr scheduling: Reorder independent instrs to avoid stalls. **Loop unrolling** enlarges scope.

CPS: Confirmed NOT on exam.

Undecidability: Alias analysis, precise liveness, optimal reg alloc all undecidable/NP-hard. Safe approximation:

- Liveness: false pos (extra pressure, correct)
- Alias: false pos (miss opts, correct)
- Reg alloc: may fail $\Rightarrow$ spill (slow, correct)

Optimization $\leftrightarrow$ Analysis

Optimization	Driven by
CSE	Available Expressions
Const/Copy propagation	Reaching Definitions
Dead code elimination	Reaching Defs + Liveness
Live range splitting	Reaching Definitions
Register allocation	Liveness
Code hoisting	Very Busy Expressions
LICM safety	Very Busy + Domination